代码主要分层：

HTTP请求
↓
Security Config
配置安全过滤链
↓
service：执行业务
↓
Mapper：访问数据库
↓
MySQL

用户表中的关键数据库约束

id
username
password_hash
phone
email

service提前检查重复，是为了给出清楚的提示

数据库唯一索引，是为了保证并发情况下也不能插入重复数据

`USER` 表示一条用户记录

例如当前实体中的字段：

```
private Long id;
private String username;
private String password_hash
```

数据库查出的记录，会被映射成 `USER` 对象：

`USER user = userMapper.selectById(userId)`

`USERMapper` 提供数据库操作

```
@Mapper
public interface USERMapper extends BaseMapper<USER>{}
```

MyBatis-Plus 提供就有操作

注册功能的完整过程

接收注册JSON
↓
转换成 RegisterRequest
↓
校验参数
↓
检查密码的哈希
↓
插入数据库
↓
返回注册成功

RegisterRequest（约束输入）：它接收的是客户端提交的数据
并且有检查数据，并且编成码
Letter对用户名、邮箱、邮箱 做这些处理
↓
AuthController（接收注册请求）：
service是业务的能力
controller是外部调用这项能力的入口
↓
AuthService（执行注册业务）：
RegisterRequest：用户提交的数据
↓
USER：注意保存的数据

将返回对象写入HTTP响应 → @RestController
定义这一组接口自动拼路径 → @RequestMapping("/api/auth")
定义注册接口 → @PostMapping("/register")
将JSON对象反序列化为 Java 对象 → @RequestBody
触发对请求对象上的参数校验 → @Valid
交给后端处理 → authService.register(request)
控制HTTP的状态信息 → ResponseEntity

基本格式校验通过，不代表业务允许注册。例如用户名格式正确，但已被别人使用
因此还要查询数据库。存在则抛出异常，不存在则校验通过后创建实体
处理密码，最后插入（insert）

petcure.BackendApplication	启动Spring Boot，让应用开始监听HTTP请求
application.yaml	主要做数据库连接，还包含JWT签发者、接收方及有效期
PasswordConfig	创建 BCryptPasswordEncoder
JwtConfig	使用环境变量中的密钥创建 JwtEncoder 和 JwtDecoder
SecurityConfig	创建安全过滤链，配置哪些接口允许访问

登录功能的完整过程

接收 LoginRequest
↓
按用户名查询用户
↓
核对密码
↓
生成 JWT
↓
更新最后登录时间
↓
返回 LoginResponse

LoginRequest 接收登录数据：

```
private String username
private String Password
```

↓

AuthService.login() 查询和核对：
查询用户：.eq(USER::getUsername)
验证身份：if (user == null || !PasswordEncoder.matches(
request.getPassword(),
user.getPassword_hash)
)
签发Jwt：验证成功后才签发JWT
返回给门的登录结果：return new LoginResponse(Jwt.getTokenValue, "Bearer", Jwt.getExpiresAt())

JWT自的三个文件分别做什么：

JwtConfig	准备签发器和验证器
JwtService	给新用户登录牌
securityConfig	规定请求什么时候必须要登录牌